

Reporte del proyecto 1 en Rust

Modelado y Programación

Rendón Ávila Jesús Mateo

27 de marzo de 2026

1. Introducción

Este reporte presenta el diseño e implementación de un sistema de chat distribuido de línea de comandos, desarrollado en Rust y organizado en dos ejecutables: `server` y `client`. El objetivo general del proyecto fue construir una aplicación funcional de mensajería en tiempo real, apoyada en redes TCP, serialización JSON y concurrencia con hilos.

El problema de base puede formularse de manera sencilla: permitir que varios usuarios, ubicados en procesos y terminales distintos, se identifiquen en un servidor central, intercambien mensajes públicos y privados, consulten estados, y se desconecten limpiamente. Aunque el enunciado parece directo, su solución requiere definir un protocolo, mantener estado compartido seguro y resolver la sincronización de múltiples conexiones concurrentes.

Desde una perspectiva de ingeniería, el proyecto no se limitó a “hacer que funcione”. También se buscó mantener una estructura modular comprensible, separar responsabilidades, documentar el protocolo y habilitar ejecución tanto local como contenerizada (Docker y Docker Compose).

2. Objetivos del proyecto

Objetivo general

Construir un chat cliente-servidor en Rust que permita comunicación en tiempo real entre usuarios mediante TCP y mensajes JSON.

Objetivos específicos

- Diseñar un protocolo de aplicación basado en mensajes con campo `type`.

- Implementar identificación de usuarios y prevención de duplicados.
- Soportar mensajería pública (broadcast) y privada (unicast).
- Permitir actualización y difusión de estado de presencia (**ACTIVE**, **AWAY**, **BUSY**).
- Implementar consulta de usuarios conectados.
- Gestionar desconexiones de forma consistente.
- Organizar el código en módulos con responsabilidades claras.
- Contenerizar cliente y servidor para facilitar despliegue y pruebas.

3. Justificación del uso de Rust

La elección de Rust respondió a criterios técnicos del dominio del problema (sistemas de red concurrentes) y a criterios formativos (aprender un lenguaje con modelo de memoria seguro sin recolector de basura). En particular, Rust resultó adecuado por los siguientes motivos:

1) Seguridad de memoria en tiempo de compilación

La propiedad más valiosa de Rust para este proyecto es su modelo de ownership, borrowing y lifetimes. En un servidor concurrente, los errores de memoria (use-after-free, double free, data races) pueden producir fallos sutiles y difíciles de depurar. Rust evita estas categorías en compilación, lo que incrementa la confiabilidad del sistema incluso en presencia de múltiples hilos.

2) Concurrencia segura y explícita

El servidor comparte estado global entre hilos (lista de usuarios y mapa de clientes conectados). Rust habilita este patrón con tipos explícitos como **ArcMutex**. Esto obliga a diseñar de forma consciente el acceso a datos compartidos y reduce condiciones de carrera.

3) Buen rendimiento para I/O de red

Aunque el proyecto no persigue optimización extrema, Rust ofrece desempeño cercano a C/C++ en aplicaciones de sistemas. Para un chat, esto se traduce en latencia baja en envío/recepción y buen manejo de múltiples sockets.

4) Bibliotecas de serialización.

Con `serde` y `serde_json`, la construcción y parseo de mensajes JSON es directa, clara y tipada. Esto simplifica la implementación del protocolo y evita errores manuales de formateo.

5) Sistema de construcción y pruebas.

Cargo unifica compilación, dependencias y ejecución de pruebas. La posibilidad de integrar tests unitarios e incluso pruebas de comportamiento con sockets reales mejora la calidad del proyecto con bajo costo de configuración.

6) Curva de Aprendizaje.

Rust tiene una curva de aprendizaje más pronunciada que alternativas como Go, especialmente en ownership y sistema de modulos compartidos. Sin embargo, ese costo se transformó en aprendizaje relevante para diseño concurrente seguro, que desde mi perspectiva, era uno de los objetivos académicos implícitos del proyecto.

Además de que permitió que pudiera comprender el protocolo JSON y como parsear texto a Datos en Rust y viceversa, por último la interconexión de ejecutables que nunca había usado anteriormente.

4. Bibliotecas y herramientas utilizadas

El proyecto se apoya deliberadamente en pocas dependencias externas. Esto simplifica mantenimiento y deja visibles los conceptos de redes implementados.

Bibliotecas de Rust

Biblioteca	Uso en el proyecto
<code>std::net</code>	<code>TcpListener</code> y <code>TcpStream</code> para servidor y cliente por sockets TCP.
<code>std::thread</code>	Creación de un hilo por conexión entrante en servidor; hilo lector en cliente.
<code>std::sync</code>	<code>Arc</code> y <code>Mutex</code> para estado compartido entre hilos; <code>mpsc</code> en cliente para sincronizar identificación.

<code>std::io</code>	Lectura/escritura por stream, <code>BufReader</code> , entrada por CLI (<code>stdin</code>).
<code>std::collections</code>	<code>HashMap</code> para usuarios conectados y asociación <code>username</code> $\rightarrow$ stream.
<code>serde</code>	Derivación de serialización/deserialización en estructuras de protocolo.
<code>serde_json</code>	Construcción de respuestas JSON con <code>json!</code> y parseo de mensajes recibidos.

Herramientas de desarrollo y despliegue

- **Cargo:** compilación, ejecución y pruebas de ambos ejecutables.
- **Docker:** generación de imágenes de runtime ligeras para cliente y servidor.
- **Docker Compose:** orquestación de servicios `chat-server` y `chat-client` para pruebas rápidas multi-contenedor.

5. Diseño general del sistema

La arquitectura corresponde a un modelo cliente-servidor clásico.

Servidor

El servidor escucha conexiones TCP en una dirección `host:port`. Por cada nueva conexión aceptada, crea un hilo dedicado que ejecuta el manejo de esa sesión. El estado global del sistema se mantiene en dos estructuras compartidas:

- **ListOfUsers:** almacena usuarios y su estado actual.
- **ConnectedClients:** mapa de `username` a stream de escritura para enviar mensajes a destinatarios específicos o realizar broadcast.

Ambas estructuras se encapsulan en `Arc<Mutex<...>`, permitiendo acceso seguro desde múltiples hilos.

Cliente

El cliente abre una conexión al servidor y se divide en dos flujos concurrentes:

- **Flujo de salida:** bucle de lectura de comandos/mensajes desde CLI y escritura al socket.
- **Flujo de entrada:** hilo lector que procesa mensajes del servidor, interpreta su tipo JSON y muestra salidas amigables en terminal.

Antes de entrar al bucle de chat, el cliente realiza `IDENTIFY` y espera la respuesta con un canal `mpsc`; si no hay `SUCCESS`, aborta la sesión.

6. Organización modular del código

Servidor: estructura y responsabilidades

- `main.rs`: inicialización, validación de argumentos, `TcpListener::bind`, creación de estado compartido e inicio de iteración de conexiones.
- `main_functions/connection_iterator.rs`: acepta conexiones entrantes y lanza un hilo por cliente.
- `main_functions/client_manager.rs`: núcleo de la lógica; parsea peticiones, despacha por `type`, responde al emisor y/o reenvía a otros.
- `main_functions/users_collection.rs`: abstracción de la colección de usuarios (alta, baja, consulta mutable e inmutable).
- `response_chat/*`: funciones puras de construcción de payloads JSON para cada operación (identificación, texto público/privado, estado, usuarios, desconexión).
- `structs_chat/user.rs`: estructuras de protocolo (por ejemplo, `Identify`) y función de parseo tipado para `IDENTIFY`.

Cliente: estructura y responsabilidades

- `main.rs`: conexión inicial, validación de username, envío de `IDENTIFY`, espera de confirmación y arranque del bucle de chat.
- `main_functions/message_loop.rs`: lectura de CLI y envío de payloads.

- `main_functions/server_reader.rs`: parseo de respuestas entrantes y presentación en terminal (incluye coloreado por usuario).
- `main_functions/usr_validation.rs`: validación local de username (no vacío, longitud máxima de 8 caracteres).
- `response_client/*`: generación de payloads para comandos `/disconnect`, `/users`, `/status`, `/w` y texto público.

7. Protocolo y operaciones implementadas

La documentación del proyecto define un protocolo JSON amplio. En el estado actual del código, se observan implementadas de manera funcional las siguientes operaciones principales:

- **IDENTIFY**: registro inicial del usuario y control de duplicados.
- **PUBLIC_TEXT**: difusión a todos los clientes excepto emisor.
- **TEXT**: envío privado a un destinatario específico.
- **USERS**: consulta de usuarios conectados y su estado.
- **STATUS**: actualización y difusión de estado del emisor.
- **DISCONNECT**: cierre de sesión y notificación a otros clientes.

Ejemplo de interacción desde cliente:

```
1 /status AWAY
2 /users
3 /w Luis Hola, estas disponible?
4 Mensaje publico para todos
5 /disconnect
```

A nivel de intercambios, los mensajes se codifican como objetos JSON terminados en salto de línea, lo cual permite usar lectura por línea con `BufRead::read_line`.

8. Decisiones de diseño relevantes

1) Un hilo por conexión

Se eligió un modelo simple: cada cliente entrante se atiende en un hilo independiente. Esta estrategia reduce complejidad conceptual al inicio y funciona bien para cargas moderadas.

2) Estado global mínimo compartido

La sincronización se concentra en dos mapas globales (usuarios y clientes), evitando dispersar locks en muchas estructuras. Esto facilita razonar sobre invariantes, por ejemplo: “si un usuario está identificado y conectado, existe en ambos mapas”.

3) Separación entre parseo, lógica y armado de respuestas

La capa `response_chat` / `response_client` mantiene funciones de serialización aisladas; `client_manager` concentra flujo del chat; y los módulos de red se enfocan en E/S. Esta separación mejora legibilidad, pruebas y evolución del protocolo.

4) Cliente con lector asíncrono y escritor interactivo

Sin esta separación, la interfaz bloquearía al esperar entrada local o mensajes remotos. Con dos flujos concurrentes se logra experiencia de chat real: se pueden recibir notificaciones mientras el usuario escribe.

9. Concurrency, seguridad y robustez

Concurrency

El uso de `Arc<Mutex<T` garantiza exclusión mutua al modificar estructuras compartidas. El diseño evita compartir referencias no sincronizadas a sockets entre hilos, encapsulando los streams en `Arc<Mutex<TcpStream` cuando deben reutilizarse para envíos.

Seguridad lógica del protocolo

El servidor valida el tipo de mensaje y, en operaciones sensibles, exige que el usuario esté previamente identificado. También normaliza valores de estado (por ejemplo, `away` → `AWAY`) y verifica existencia de destinatarios en mensajes privados.

Manejo de fallas

Se contemplan errores comunes: dirección en uso al iniciar servidor, fallo de clonado de stream, errores de lectura/escritura y JSON inválido. En muchos casos se evita terminar el proceso completo y se limita el impacto a la sesión afectada.

10. Pruebas y validación

Una parte importante del proyecto es la verificación. El código incluye pruebas unitarias y de integración ligera con sockets reales en memoria local.

Casos verificados

- Correctitud de JSON generado para IDENTIFY.
- Respuesta NO_SUCH_USER en mensaje privado a usuario inexistente.
- Entrega efectiva de TEXT_FROM cuando destinatario está conectado.
- Actualización de estado y difusión de NEW_STATUS.
- Comando de cliente /disconnect mapeado correctamente a “payload”DISCONNECT.

Estas pruebas son valiosas porque no sólo validan funciones aisladas, sino también interacciones de red y consistencia del protocolo serializado.

11. Despliegue con Docker

El proyecto incluye `Dockerfile` para cliente y servidor con construcción por etapas (multi-stage):

- Etapa `builder`: compila en imagen oficial de Rust.
- Etapa `runtime`: copia sólo el binario final sobre Debian slim, reduciendo tamaño de imagen.

Con `docker-compose.yml` se levantan ambos servicios de forma simple. Además, se parametriza la dirección de escucha/conexión mediante variables de entorno (`CHAT_BIND_ADDR`, `CHAT_SERVER_ADDR`), lo que facilita pruebas en distintos entornos sin recompilar.

12. Alcance actual y trabajo pendiente

El protocolo documentado contempla funciones avanzadas de cuartos (`NEW_ROOM`, `INVITE`, `JOIN_ROOM`, `ROOM_TEXT`, etc.). En la implementación actual del servidor, el núcleo funcional prioriza mensajería global debido a la falta de tiempo y a la complejidad encontrada durante el proceso de programación del proyecto, aún así, se logró de la manera que pareció mas

adecuada conexiones privadas, presencia de usuarios y desconexión. Esto es una decisión de alcance para una entrega inconclusa: primero consolidar base estable y posteriormente extender.

Siguientes mejoras por aplicar:

- Implementar completamente la capa de cuartos conforme al protocolo.
- Estandarizar respuestas de error inválido/no identificado de forma central.
- Incorporar pruebas end-to-end multiusuario automatizadas.
- Considerar un modelo asíncrono (por ejemplo, con runtime async) para escalar mejor a gran número de conexiones.

13. Conclusiones

El Chat_Project demuestra que Rust es una opción técnicamente sólida para aplicaciones cliente-servidor concurrentes incluso en proyectos académicos. La combinación de seguridad de memoria, control de concurrencia y ecosistema de serialización permitió construir un sistema funcional, modular y desplegable.

Desde el punto de vista de diseño, la separación entre servidor y cliente, el despacho por tipos de mensaje, y la organización por módulos facilitan tanto la lectura del código como su evolución. Asimismo, la inclusión de pruebas y Docker acerca el proyecto a prácticas reales de ingeniería.

En síntesis, el proyecto cumple con los objetivos fundamentales de comunicación en tiempo real por CLI y establece una base robusta para crecer hacia funciones más avanzadas del protocolo, especialmente gestión de cuartos y validación de errores más estricta.